

PROJECT AND TEAM INFORMATION

Project Title

Text - Editor

Team Name: *Team #*	*DSCPP-III-2026-T185*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Joshi, Arinjay – 2510012368* *2510012268@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Mandal, Arunabha – 2510011661* *2510011661@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Datta, Mahek – 2510011659* *2510011659@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Dastidar, Mehuli – 2510011660* *2510011660@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract

This project aims to develop a GUI-based multi-line text editor using C++ and Data Structures. Nested doubly linked lists are used to manage multi-line text and cursor movement, while a skip list supports faster upward and downward navigation. The ‘gotoxy()‘ function positions the cursor within the terminal, and the ‘TextEditor‘ class manages operations such as cursor movement, text insertion, and deletion. The project demonstrates the practical application of linked lists, skip lists, pointers, and Object-Oriented Programming concepts to build an interactive text editor.

Updated Project Approach and Architecture

The project follows a structured approach to develop a console-based multi-line text editor using C++ and Data Structures. Nested doubly linked lists are used to organize and manage multi-line text, enabling navigation between characters and lines. A skip list is incorporated to support faster upward and downward cursor movement across lines. The ‘TextEditor‘ class encapsulates the primary editing operations, including cursor movement, text insertion, and deletion, ensuring modularity and maintainability. The ‘gotoxy()‘ function is used to position the visible cursor at specific coordinates within the terminal, keeping cursor navigation interactive. The implementation combines linked lists, pointers, functions, and object-oriented programming concepts to manage text and editing operations. The development process includes designing the data structures, implementing the editor functions, integrating cursor navigation with text manipulation, and testing the system for correct multi-line handling, insertion, deletion, and movement. This approach demonstrates the practical application of Data Structures and OOP concepts in building an interactive text editor. .

Tasks Completed

Task Completed	Team Member
Finalized project topic and scope; identified requirements and core features; selected nested doubly linked lists for multi-line text and cursor movement; planned skip-list-based vertical navigation; designed the TextEditor class and identified implementation milestones, testing, and implementation strategy.	Team Avengers (all members; assign individual ownership during implementation)

Challenges/Roadblocks

The main technical challenge is keeping the cursor position consistent with the linked-list structure when inserting or deleting characters, especially at the beginning or end of a line. Multi-line editing also requires correct handling of line boundaries and movement between adjacent lines. The skip list must remain synchronized with the line structure when lines are added or removed. Terminal cursor positioning through gotoxy() can vary by platform. These challenges will be addressed by validating pointer updates after every edit, testing boundary cases, updating skip-list links whenever the line structure changes, and testing terminal behavior in the intended environment.

Tasks Pending

Task Pending	Team Member (to complete the task)
Integrate all data structures; complete cursor navigation, insertion and deletion; test boundary cases; validate skip-list updates; finalize documentation and demonstration.	Team Lead.

Project Outcome/Deliverables

The main outcome of this project is to develop a GUI-based multi-line text editor using C++ and Data Structures. It will support text insertion, deletion, and cursor navigation. A nested doubly linked list will manage text and cursor movement, while a skip list will improve vertical navigation between lines. The 'TextEditor' class will organize these operations into a modular structure. This project demonstrates the practical application of data structures and object-oriented programming in text-editing software.

Progress Overview

The project topic and scope have been finalized. Requirements and core features have been identified, and the required data structures and system architecture have been planned. The design uses a nested doubly linked list for multi-line text and cursor movement, a skip list for faster up/down navigation, gotoxy() for terminal cursor positioning, and a TextEditor class for cursor movement, insertion, and deletion. Development milestones and a testing/implementation strategy have been defined. Remaining work is to complete integration, verify all operations, and record actual test results. The completion percentage should be updated after implementation.

Codebase Information

Repository: [https://github.com/arinjayjoshi224-cell/Text-Editor-in-CPP]
Branch: Main
Important commits: gotoxy and skip list added and Create typing engine.cpp.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Multi-line Text Management	Pass	Nested doubly linked list functionality checked.
Cursor Navigation	Pass	Cursor movement using skip list and gotoxy() checked.

Deliverables Progress

- GUI design and development for the multi-line text editor.
- Implementation of nested doubly linked lists for multi-line text management.
- Implementation of skip-list-based vertical cursor navigation and cursor positioning using `gotoxy()`.
- Development of the `TextEditor` class with functions for cursor movement, text insertion, and deletion.
- Initial integration of data structures and core editing operations.